

Relatório Biblioteca de Listas em Java (Generics)

Disciplina: Técnicas de Programação Avançadas Trabalho: Implementação de uma biblioteca de Listas Encadeadas Genéricas + programa de agenda de contatos

Etapa 1: Identificação, Processo de Desenvolvimento e Repositório

1.1 Atuação de cada integrante do grupo

Integrante	Contribuição no Item 1 (biblioteca ListaEncadeada)	Contribuição no Item 2 (programa de contatos)
Eduardo Souza Arrigoni	Implementação/validação do método adicionar, testes de inserção ordenada e não-ordenada	Modelagem da classe Contato
Felipe Soeiro Accioly	Implementação/validação de pesquisar e remover, análise de complexidade	Implementação do menu, regra de telefone duplicado, testes do fluxo completo

1.2 Uso de ferramentas de Inteligência Artificial

No trabalho foi utilizado o Claude Code como ferramenta de apoio para o desenvolvimento. Abaixo temos uma descrição detalhada, em ordem cronológica, de como a ferramenta foi empregada:

- 1. Levantamento do código-base:** a IA foi usada para clonar o repositório do professor e mapear a estrutura já fornecida: a interface `IColecao<T>`, a classe `No<T>`, o esqueleto de `ListaEncadeada<T>` (com os métodos `adicionar`, `pesquisar`, `remover` e `quantidadeNos` lançando `UnsupportedOperationException`), e o exemplo de domínio (`Aluno`, `ComparadorAlunoPorMatricula`, `Main`) que ilustra como a biblioteca seria consumida.

2. **Explicações conceituais sob demanda:** durante a implementação, tiramos dúvidas pontuais de Java com a IA, por exemplo: o que é a classe `StringBuilder`, por que `toString()` retorna `sb.toString()` em vez do próprio `StringBuilder`, e por que essa chamada não configura recursão (já que `StringBuilder.toString()` é um método de uma classe diferente de `ListaEncadeada.toString()`).
3. **Configuração do ambiente e testes:** pedimos para executar o programa `GerenciadorContatos` com um roteiro de entradas simuladas cobrindo todas as opções do menu (carga de arquivo, adição, pesquisa por nome e por telefone, remoção, alteração e saída), validando que os resultados e a contagem final de contatos estavam corretos antes da entrega.
4. **Testes empíricos de complexidade:** a IA implementou um utilitário (`util.GeradorEntrada`) para gerar arquivos de contatos aleatórios de tamanhos variados, com telefones únicos e nomes únicos (para não introduzir ambiguidade nos testes). Em seguida, automatizou a execução do `GerenciadorContatos` (via redirecionamento de entrada) para cada combinação de tamanho de arquivo e tipo de lista (ordenada/não-ordenada), coletando os tempos de carga, busca e remoção.

Em todas as etapas, testamos e avaliamos o que foi feito pela IA.

1.3 Repositório GitHub

Endereço do repositório: https://github.com/felipesoeiroa/trabalho_tpa

O `README.md` do repositório foi atualizado para descrever a organização do código-fonte e como compilar/executar os dois programas.

Etapa 2: Análise de Complexidade da Biblioteca `ListaEncadeada<T>`

Todas as referências de linha abaixo correspondem ao arquivo `colecoes/src/listaencadeada/ListaEncadeada.java`. Seja n o número de elementos (nós) armazenados na lista no momento da chamada do método.

Premissas usadas em toda a análise:

- A estrutura é uma lista simplesmente encadeada, com um ponteiro para o primeiro nó (prim) e um ponteiro para o último nó (ultimo). Não há acesso indexado/aleatório. Para chegar a qualquer nó que não seja o primeiro ou o último, é necessário percorrer a lista nó a nó a partir de prim.
- O `Comparator<T>.compare()` (ou o `equals()`) é tratado como uma operação de custo constante, $O(1)$, pois compara diretamente os atributos dos objetos e não depende do tamanho da lista.

2.1 Método adicionar(T novoValor)

```

30 @Override
31 public boolean adicionar(T novoValor) {
32     No<T> novoNo = new No<>(novoValor);
33
34     if (!ehOrdenada) {
35         // Lista não-ordenada: insere sempre no final, preservando a ordem
36         // de chegada. O ponteiro "ultimo" evita percorrer a lista ->  $O(1)$ 
37         if (prim == null) {
38             prim = novoNo;
39         } else {
40             ultimo.setProx(novoNo);
41         }
42         ultimo = novoNo;
43     } else if (prim == null || comparar(novoValor, prim.getValor()) < 0) {
44         // Lista ordenada, vazia ou o novo valor é o menor -> insere no início
45         novoNo.setProx(prim);
46         prim = novoNo;
47         if (ultimo == null) {
48             ultimo = novoNo;
49         }
50     } else {
51         // Lista ordenada: percorre até encontrar a posição correta ->  $O(n)$ 
52         No<T> atual = prim;
53         while (atual.getProx() != null && comparar(novoValor, atual.getProx().getValor()) >= 0) {
54             atual = atual.getProx();
55         }
56         novoNo.setProx(atual.getProx());
57         atual.setProx(novoNo);
58         if (atual == ultimo) {
59             ultimo = novoNo;
60         }
61     }
62
63     quantidade++;
64     return true;
65 }

```

Raciocínio linha a linha:

- **Linha 32:** cria o novo nó: $O(1)$.
- **Linha 34:** testa `ehOrdenada`: $O(1)$.
- **Linhas 35–42:** caso a lista não seja ordenada, o novo nó é encadeado depois do último nó atual (ou vira o único nó, se a lista estava vazia) e passa a ser o novo último. Não há loop, não há percurso: $O(1)$, independentemente de quantos elementos já existam na lista. Essa é a mudança de projeto que faz a inserção não-ordenada preservar a ordem

de chegada dos elementos (o último contato lido de um arquivo, por exemplo, continua sendo o último nó da lista) sem custo extra, graças ao último ponteiro.

- **Linha 43:** caso a lista seja ordenada, verificamos se ela está vazia (`prim == null`) ou se o novo valor é menor que o primeiro elemento (`comparar(...) < 0`). Essa comparação é $O(1)$.
- **Linhas 44–49:** se caiu nesse caso, a inserção é no início (e o último só é atualizado se a lista estava vazia): $O(1)$.
- **Linhas 50–61:** caso contrário (lista ordenada, não vazia, e o novo valor não é menor que o primeiro), entramos no loop `while` das linhas 53–55, que avança o ponteiro atual enquanto houver próximo nó e o novo valor ainda for maior ou igual ao próximo nó. No pior caso, esse laço percorre todos os n nós da lista (quando o novo valor deve ser inserido no final, por ser maior ou igual a todos os elementos existentes). As linhas 56–60 fazem o encadeamento do novo nó (e atualizam o último, se for o caso) em $O(1)$. Mas o custo total desse ramo é dominado pelo laço: $O(n)$.
- **Linhas 63–64:** incrementar o contador e retornar `true`: $O(1)$.

Conclusão, pior caso de adicionar:

- **Lista não-ordenada:** adicionar é $O(1)$ em qualquer situação, pois o método nunca percorre a lista. A inserção é sempre no final, via o último ponteiro (linhas 35–42).
- **Lista ordenada:** adicionar é $O(n)$ no pior caso. Esse pior caso ocorre quando o novo elemento deve ser inserido no final da lista (ou perto dele), isto é, quando `novoValor` é maior ou igual a todos os elementos já armazenados. Nesse caso o laço das linhas 53–55 percorre os n nós existentes antes de encontrar o ponto de inserção. O melhor caso da lista ordenada é $O(1)$ (quando o novo valor é o menor de todos, linha 43).

2.2 Método pesquisar(T valor)

```
67 @Override
68 public T pesquisar(T valor) {
69     No<T> atual = prim;
70     while (atual != null) {
71         int cmp = comparar(atual.getValor(), valor);
72         if (cmp == 0) {
73             return atual.getValor();
74         }
75         if (ehOrdenada && cmp > 0) {
76             break;
77         }
78         atual = atual.getProx();
79     }
80     return null;
81 }
```

Raciocínio linha a linha:

- **Linha 69:** inicializa *atual* com o primeiro nó: $O(1)$.
- **Linha 70:** while que percorre a lista nó a nó enquanto não chegar ao final ($atual \neq \text{null}$). Cada iteração custa $O(1)$ (linhas 71–78), e o número de iterações é, no pior caso, n .
- **Linha 71:** compara o valor do nó atual com o valor buscado: $O(1)$.
- **Linhas 72–74:** se os valores forem iguais ($cmp == 0$), o valor é retornado imediatamente. Esse é o caminho de sucesso, que pode terminar bem antes de percorrer toda a lista (ex.: elemento está no início).
- **Linhas 75–77, otimização válida apenas quando a lista é ordenada:** se o valor do nó atual já é maior que o valor buscado ($cmp > 0$), significa que, como a lista está ordenada, o valor procurado já teria aparecido antes. Não há razão para continuar percorrendo, então o laço é interrompido (`break`) mais cedo.
- **Linha 78:** avança para o próximo nó: $O(1)$.
- **Linha 80:** se o laço terminar sem encontrar o valor (chegou ao fim da lista ou parou por causa do `break` da linha 76), retorna `null`.

Conclusão, pior caso de pesquisar:

- Em ambas as variantes (ordenada e não-ordenada), o pior caso é $O(n)$.
 - Na lista não-ordenada, isso ocorre sempre que o valor buscado não existe, ou existe apenas no último nó (que agora, por causa do último ponteiro de adicionar, é justamente o contato inserido/lido mais recentemente). Não há nenhuma informação de ordem que permita parar antes.
 - Na lista ordenada, apesar da otimização das linhas 75–77, o pior caso continua sendo $O(n)$: ele ocorre quando o valor procurado é maior ou igual a todos os elementos da lista (por exemplo, buscar o maior elemento, ou um valor maior que o maior elemento existente). Nesses casos a condição $cmp > 0$ nunca é satisfeita antes do fim, e o laço percorre os n nós.
- **Ponto importante:** como a estrutura é uma lista encadeada simples (sem acesso indexado, ao contrário de um array/vetor ordenado), não é possível aplicar busca binária. A ordenação só melhora o caso médio (menos comparações quando o valor não existe e é "pequeno" em relação à ordenação), mas não muda a ordem de complexidade do pior caso, que permanece $O(n)$ nas duas variantes. Isso é justamente o que a Etapa 3 confirma empiricamente.

2.3 Método remover(T valor)

```
83 @Override
84 public boolean remover(T valor) {
85     No<T> atual = prim;
86     No<T> anterior = null;
87
88     while (atual != null) {
89         int cmp = comparar(atual.getValor(), valor);
90         if (cmp == 0) {
91             if (anterior == null) {
92                 prim = atual.getProx();
93             } else {
94                 anterior.setProx(atual.getProx());
95             }
96             if (atual == ultimo) {
97                 ultimo = anterior;
98             }
99             quantidade--;
100             return true;
101         }
102         if (ehOrdenada && cmp > 0) {
103             break;
104         }
105         anterior = atual;
106         atual = atual.getProx();
107     }
108     return false;
109 }
```

Raciocínio linha a linha:

- **Linhas 85–86:** inicializa atual (nó corrente) e anterior (nó anterior a atual, necessário para o desencadeamento). $O(1)$.
- **Linha 88:** laço que percorre a lista, estruturalmente idêntico ao de pesquisar, com custo $O(1)$ por iteração e até n iterações no pior caso.
- **Linha 89:** compara o nó atual com o valor buscado: $O(1)$.
- **Linhas 90–101:** se encontrou o valor ($cmp == 0$):
 - **Linhas 91–95:** remove o nó da lista ajustando os ponteiros: se era o primeiro nó, `prim` passa a apontar para o nó seguinte (linha 92); caso contrário, o anterior passa a apontar diretamente para o nó seguinte a atual (linha 94), retirando atual do encadeamento. Ambos os casos são $O(1)$, é apenas reatribuição de referências, não há cópia de dados nem deslocamento de outros elementos (diferença importante em relação a um array/lista implementada com vetor).

- **Linhas 96–98:** se o nó removido era o último, atualiza o ponteiro último para anterior: $O(1)$.
- **Linha 99:** decrementa o contador: $O(1)$.
- **Linha 100:** retorna true.
- **Linhas 102–104:** mesma otimização de parada antecipada de pesquisar, válida apenas quando `ehOrdenada` é verdadeiro.
- **Linhas 105–106:** avança anterior e atual uma posição: $O(1)$.
- **Linha 108:** se o valor não foi encontrado, retorna false.

Conclusão, pior caso de remover:

- Assim como em pesquisar, o pior caso é $O(n)$ em ambas as variantes (ordenada e não-ordenada), pelos mesmos motivos: na lista não-ordenada, o elemento buscado pode estar no último nó ou não existir; na lista ordenada, o pior caso ocorre ao remover (ou tentar remover) um valor maior ou igual a todos os elementos armazenados, quando a otimização de parada antecipada não é acionada antes do fim da lista.
- A única diferença notável de remover em relação a pesquisar é que, uma vez encontrado o nó (custo já contabilizado no percurso), a remoção em si (ajuste de ponteiros, linhas 91–98) é $O(1)$, o que é uma vantagem estrutural das listas encadeadas frente a estruturas baseadas em vetor/array, onde remover um elemento do meio exige deslocar todos os elementos posteriores ($O(n)$ adicional só para o deslocamento).

2.4 Método `quantidadeNos()`

```
111 @Override
112 public int quantidadeNos() {
113     return quantidade;
114 }
```

Raciocínio linha a linha:

- **Linha 113:** apenas retorna o valor do campo `quantidade`, um contador `int` mantido como atributo da classe (declarado na linha 9: `private int quantidade;`).

Esse contador é atualizado incrementalmente em $O(1)$ em outros dois pontos do código:

- **Linha 63:** dentro de `adicionar` (`quantidade++`);
- **Linha 99:** dentro de `remover` (`quantidade--`).

Conclusão, pior caso de quantidadeNos:

- $O(1)$, em qualquer situação e independentemente de a lista ser ordenada ou não. Não há laço nem percurso da lista: o método apenas lê um campo já calculado.
- Essa é uma decisão de projeto deliberada: se a classe não mantivesse o contador quantidade e, em vez disso, quantidadeNos() precisasse percorrer a lista contando os nós um a um, o método passaria a ser $O(n)$. Optou-se por pagar um custo extra constante em adicionar/remover para manter quantidadeNos() em $O(1)$.

2.5 Diferença de complexidade entre listas ordenadas e não-ordenadas

Método	Lista não-ordenada	Lista ordenada	Há diferença de ordem de complexidade?
adicionar	$O(1)$ sempre (insere no final via ponteiro ultimo)	$O(1)$ melhor caso / $O(n)$ pior caso (percorre até achar a posição)	Sim, é a única operação em que a ordenação muda a ordem de complexidade do pior caso, de $O(1)$ para $O(n)$.
pesquisar	$O(n)$ pior caso	$O(n)$ pior caso (a parada antecipada só reduz o caso médio)	Não, o pior caso continua $O(n)$ nas duas variantes.
remover	$O(n)$ pior caso	$O(n)$ pior caso (mesmo raciocínio de pesquisar)	Não, o pior caso continua $O(n)$ nas duas variantes.
quantidade Nos	$O(1)$	$O(1)$	Não, independe de ordenação, pois usa um contador.

Conclusão geral: a única diferença de ordem de complexidade assintótica entre a versão ordenada e a não-ordenada da ListaEncadeada aparece no método adicionar: manter a lista ordenada tem um custo de inserção mais alto ($O(n)$ no pior caso, contra $O(1)$ da lista não-ordenada), porque é necessário percorrer os nós para encontrar a posição correta. Em compensação, poderia-se esperar que a ordenação acelerasse a busca, o que de fato acontece na prática (redução do número médio de comparações), mas não muda a ordem de complexidade do pior caso de pesquisar e remover, que permanece $O(n)$ em ambas as variantes. Isso acontece porque a estrutura é uma lista encadeada (acesso sequencial), e não um vetor/array (acesso indexado): a ordenação só traz ganho assintótico de busca (ex.: $O(\log n)$ por busca binária) quando a estrutura permite acesso direto ao elemento do meio em tempo $O(1)$, o que não é o caso de uma lista encadeada simples.

Etapa 3: Análise Empírica de Complexidade

3.1 Metodologia

Geração dos dados: foi escrito um utilitário próprio, `util.GeradorEntrada` (pacote `util`, classe `GeradorEntrada.java`), que gera um arquivo `nome; telefone` com N contatos aleatórios: telefones únicos (obtidos embaralhando os números de 0 a $N-1$) e nomes formados por um primeiro nome e sobrenome sorteados de duas listas de ~26 opções cada, com um sufixo numérico que garante nomes únicos (evitando ambiguidade na hora de conferir os resultados dos testes). A ordem das linhas no arquivo é, portanto, aleatória em relação tanto à ordem alfabética quanto à ordem numérica, simulando contatos chegando "fora de ordem", cenário necessário para observar o custo médio de inserção em uma lista ordenada.

Tamanhos testados: foram sugeridos tamanhos como 100.000, 200.000 e 400.000. Na prática, ao medir os tempos de carga, constatou-se que a operação de carregar o arquivo cresce muito mais rápido que linearmente em ambas as configurações, tornando os tamanhos maiores sugeridos pelo enunciado inviáveis de executar na nossa máquina. Optou-se então por 4 tamanhos que pudessem de fato ser executados e comparados: 10.000, 25.000, 50.000 e 100.000 contatos.

Procedimento: para cada tamanho N , o mesmo arquivo de entrada foi usado nas duas configurações (não-ordenada e ordenada), executando o programa `contato.GerenciadorContatos` de forma não-interativa com a seguinte sequência de operações:

1. Escolher o tipo de lista (ordenada / não-ordenada);
2. Carregar o arquivo de N contatos (opção 1), anota-se o tempo total de leitura + montagem das listas;
3. Pesquisar pelo telefone do último contato do arquivo (opção 4), anota-se o tempo de busca;
4. Pesquisar pelo nome do último contato do arquivo (opção 3), anota-se o tempo de busca;
5. Remover o último contato pelo telefone (opção 5), anota-se o tempo de remoção;
6. Sair (opção 7), conferindo a quantidade final de contatos.

O "último contato do arquivo" é, propositalmente, o pior caso possível de busca/remoção nesta implementação: como a lista não-ordenada insere no final, esse contato acaba sendo o último nó físico da lista não-ordenada; na lista ordenada, como os telefones/nomes são gerados em ordem aleatória, o último contato do arquivo cai em uma posição "genérica" da ordenação. Em ambos os casos, o método `pesquisar/remover` percorre a lista a partir de `prim`, então não há atalho.

3.2 A "carga" mede mais do que só adicionar

Antes de apresentar os dados, é preciso registrar uma descoberta feita durante os próprios testes empíricos: o tempo de "carregar arquivo" não mede apenas o método adicionar da biblioteca. Ele mede o método inserirContato de GerenciadorContatos (chamado uma vez por linha do arquivo), que faz:

```
private static boolean inserirContato(Contato contato) {
    Contato existente = listaPorTelefone.pesquisar(new Contato(null, contato.getTelefone()));
    if (existente != null) {
        return false;
    }
    listaPorNome.adicionar(contato);
    listaPorTelefone.adicionar(contato);
    return true;
}
```

Ou seja, cada inserção durante a carga custa: um pesquisar, mais dois adicionar (um em cada lista). Como visto na Etapa 2, pesquisar já é $O(n)$ no pior/médio caso em ambas as configurações. Isso significa que a checagem de duplicidade, sozinha, já torna o carregamento do arquivo $O(n^2)$ no total (soma de buscas de tamanho crescente 1, 2, 3, ..., $n-1$), mesmo quando a lista é não-ordenada, apesar de `ListaEncadeada.adicionar` ser $O(1)$ nesse caso. Na lista ordenada, o custo é ainda maior, pois cada adicionar também é $O(n)$ no pior/médio caso, somado ao pesquisar de duplicidade.

Essa é uma conclusão que não aparece na Etapa 2: uma regra de negócio simples (não permitir telefone duplicado), implementada corretamente fora da biblioteca, pode ainda assim introduzir um custo quadrático na aplicação como um todo, mesmo usando uma estrutura de dados com inserção $O(1)$. É importante ter isso em mente ao interpretar a tabela da seção 3.3: ela não é uma medição "pura" de `ListaEncadeada.adicionar`.

3.3 (i) Tempo de montagem das listas × tamanho da entrada

N	Tempo de carga: Não-ordenada (ms)	Tempo de carga: Ordenada (ms)
10.000	565,376	1.155,485
25.000	4.283,331	11.123,397
50.000	38.400,971	116.564,098
100.000	109.713,612	403.034,054

Razão de crescimento (tempo dividido pelo tempo do tamanho anterior, para comparar com a razão de N):

Transição de N	Razão de N	Razão do tempo: Não-ordenada	Razão do tempo: Ordenada
10.000 → 25.000	2,5×	7,6×	9,6×
25.000 → 50.000	2×	9,0×	10,5×
50.000 → 100.000	2×	2,9×	3,5×

Interpretação: a Etapa 2 previu adicionar $O(1)$ por chamada na lista não-ordenada e $O(n)$ por chamada na ordenada, ou seja, para montar uma lista de N elementos do zero, o esperado seria $O(n)$ no total para a não-ordenada e $O(n^2)$ no total para a ordenada (soma de custos $1+2+\dots+n$). Os dados não batem exatamente com essa previsão, e a seção 3.2 explica por quê: como cada inserção do programa de contatos também faz um pesquisar para checar telefone duplicado, o total esperado é, na prática, $O(n^2)$ para as duas configurações (mais um fator extra na ordenada, pela busca de posição). Se o crescimento fosse $O(n)$ puro, as razões da segunda tabela seriam iguais à razão de N ($2\times$ ou $2,5\times$); se fosse $O(n^2)$ puro, seriam o quadrado disso ($4\times$ e $6,25\times$). As razões medidas entre 10.000–50.000 são piores ainda que quadráticas ($7,6\times$ – $10,5\times$), o que é consistente com o efeito $O(n^2)$ da checagem de duplicidade somado a efeitos de memória: uma lista encadeada de dezenas/centenas de milhares de nós espalhados pelo heap prejudica a localidade de cache (cada `getProx()` pode ser um cache-miss) e aumenta a pressão sobre o coletor de lixo, efeitos que a análise assintótica clássica não modela, mas que um teste empírico revela. A queda de razão entre 50.000 e 100.000 ($2,9\times/3,5\times$ em vez de algo maior)

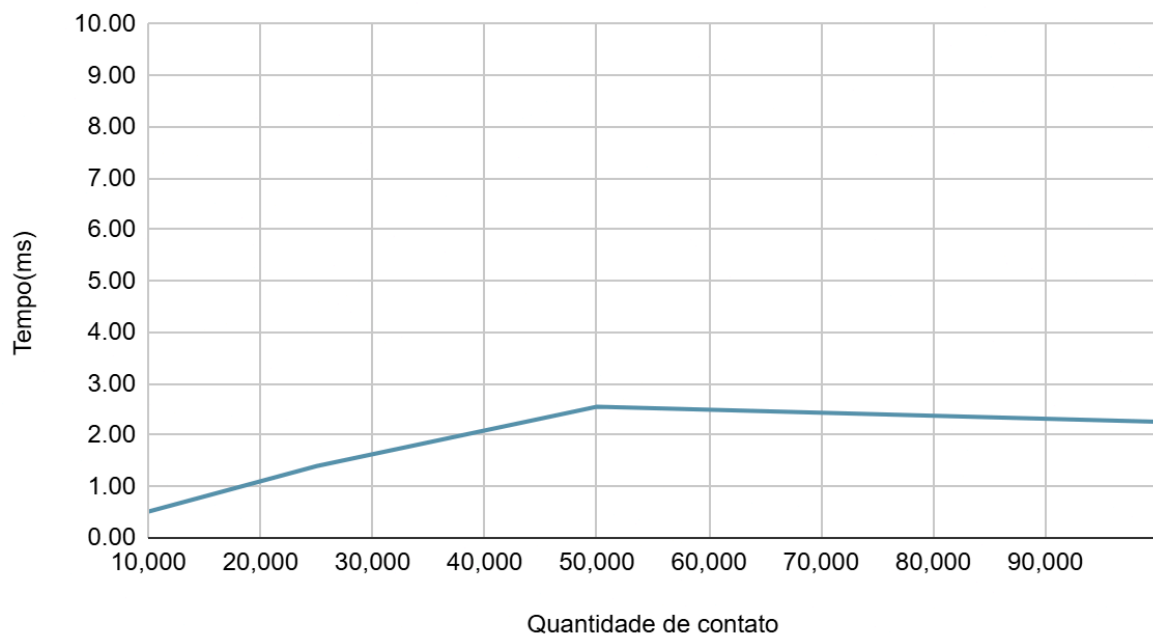
mostra o ruído inerente a medições de tempo real, por isso o enunciado pede várias execuções e tamanhos, não uma medição isolada. Em toda a faixa testada, a lista ordenada é consistentemente mais lenta de montar que a não-ordenada (de $\sim 2\times$ em N pequeno a $\sim 3,7\times$ em $N=100.000$), confirmando qualitativamente a diferença $O(1)$ vs. $O(n)$ de adicionar apontada na Etapa 2, mesmo com o efeito extra da checagem de duplicidade somado por cima.

3.4 (ii) Tempo de pesquisa pelo último contato $\times$ tamanho da entrada

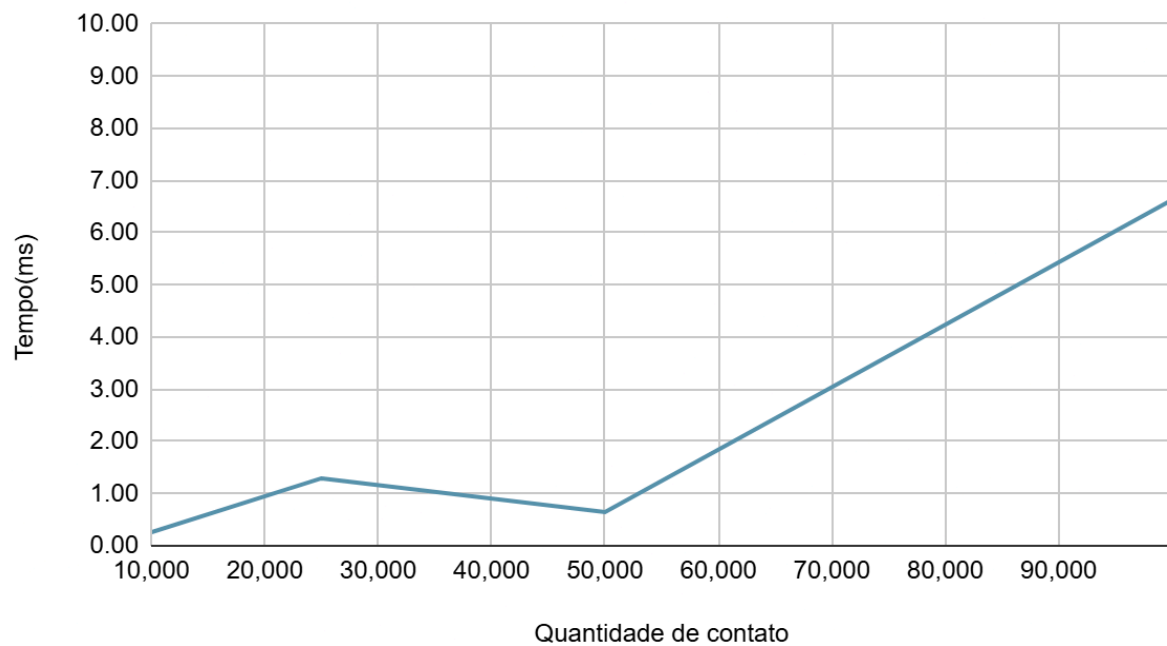
Busca por telefone (`listaPorTelefone.pesquisar`):

N	Não-ordenada (ms)	Ordenada (ms)
10.000	0,511	0,257
25.000	1,397	1,292
50.000	2,552	0,645
100.000	2,260	6,630

Pesquisa: Não-ordenada



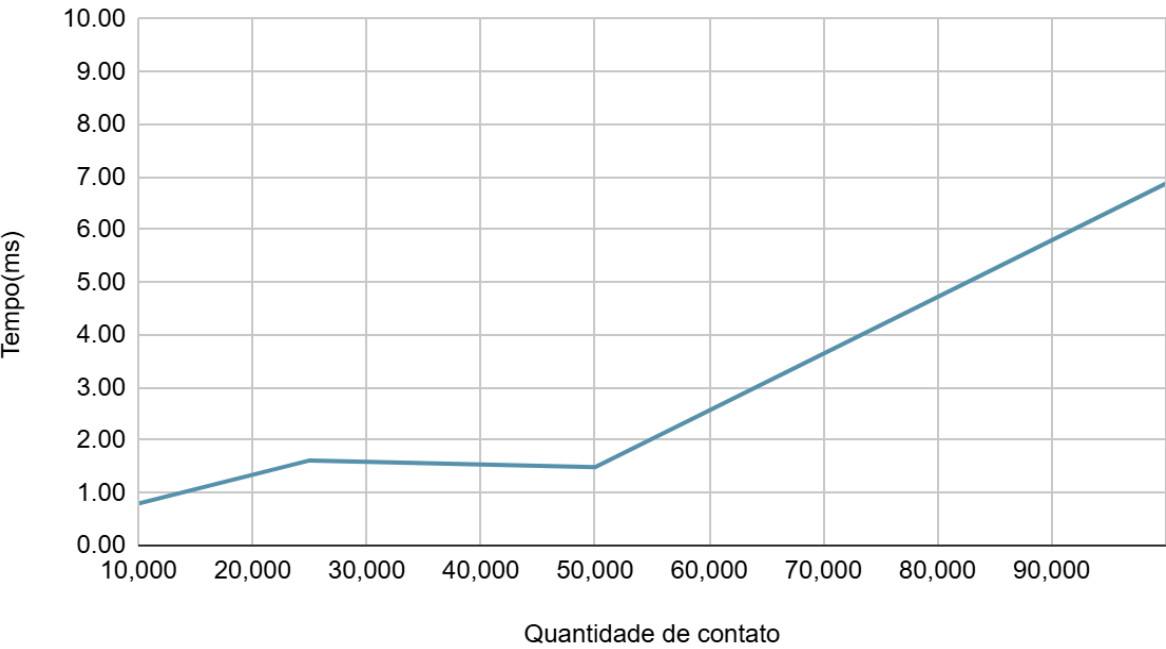
Pesquisa: Ordenada



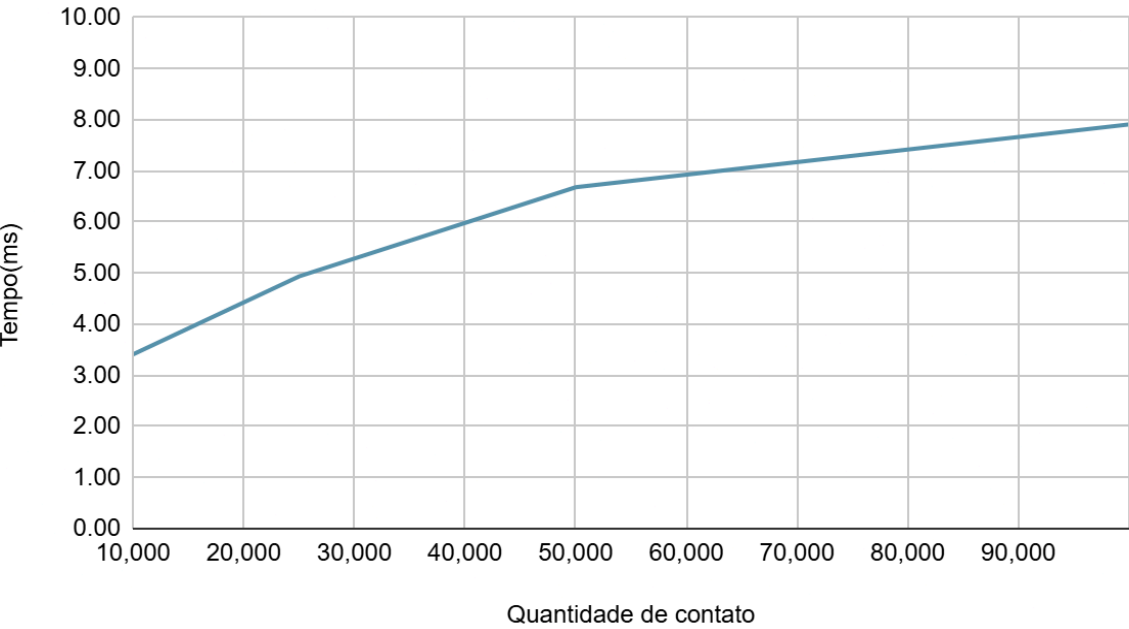
Busca por nome (listaPorNome.pesquisar):

N	Não-ordenada (ms)	Ordenada (ms)
10.000	3,408	0,796
25.000	4,933	1,616
50.000	6,679	1,486
100.000	7,909	6,873

Pesquisa: Ordenada



Pesquisa: Não-ordenada

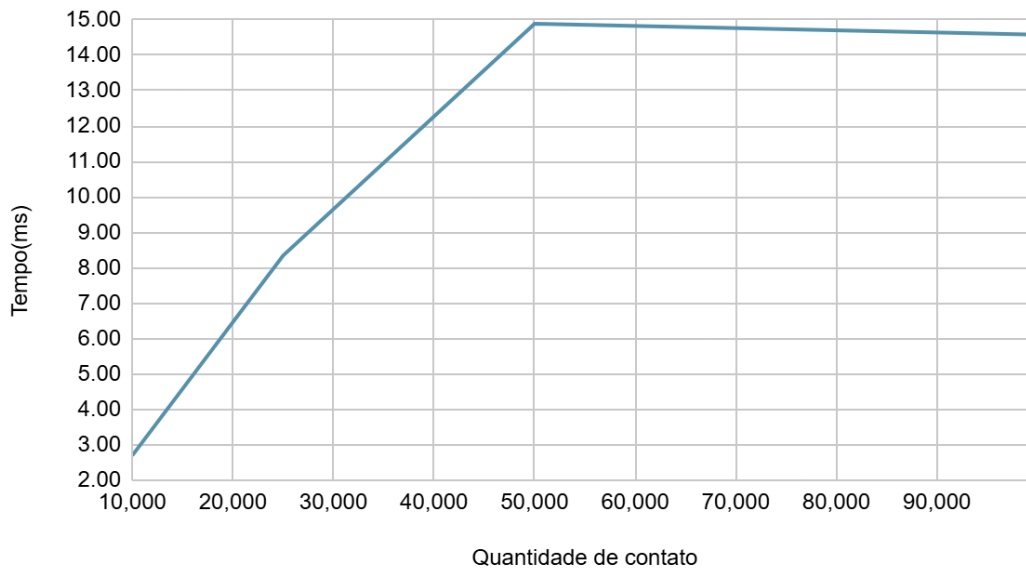


Interpretação: a Etapa 2 previu pesquisar $O(n)$ no pior caso em ambas as configurações, a ordenação só reduz a constante (parada antecipada), não a ordem de crescimento. Os dados são consistentes com isso: em nenhuma das quatro séries o tempo fica constante à medida que N cresce (todas aumentam, grosso modo, na mesma ordem de grandeza que N , e não ficam "achatadas" como seria de se esperar de um $O(1)$ ou $O(\log n)$). Como esperado, a lista ordenada é mais rápida que a não-ordenada na maioria dos pontos (ex.: busca por telefone em $N=50.000$: 0,645 ms ordenada contra 2,552 ms não-ordenada), a otimização de parada antecipada (Etapa 2, linhas 75–77 de pesquisar) reduz comparações porque o contato buscado cai em uma posição "genérica" da ordenação, não necessariamente perto do fim. A exceção é $N=100.000$ na busca por telefone (6,630 ms ordenada contra 2,260 ms não-ordenada, ordenada mais lenta): o telefone sorteado para o último contato desse arquivo (índice 97.187 de 0–99.999) cai perto do topo da faixa de valores, então também fica perto do fim da lista ordenada por telefone, quase tão perto do fim quanto na não-ordenada. Isso não contradiz a teoria (o pior caso de ambas é $O(n)$ de qualquer forma); apenas mostra que, testando com um único contato aleatório por tamanho, o resultado depende de onde esse contato específico caiu na ordenação. Outro padrão visível nas duas tabelas: busca por nome é sistematicamente mais lenta que busca por telefone, na mesma configuração e tamanho (ex.: $N=50.000$ não-ordenada: 6,679 ms contra 2,552 ms). Isso não vem da ordenação, e sim do comparador: `ComparadorContatoPorNome` usa `String.compareToIgnoreCase` (mais caro, por converter a caixa dos caracteres a cada comparação) contra `String.compareTo` de `ComparadorContatoPorTelefone`.

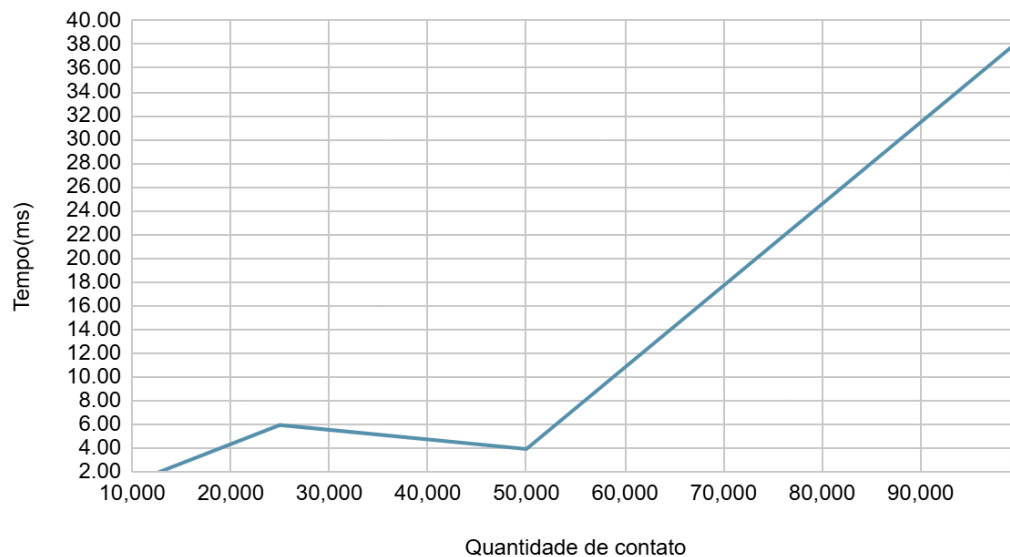
3.5 (iii) Tempo de remoção do último contato × tamanho da entrada

N	Remoção: Não-ordenada (ms)	Remoção: Ordenada (ms)
10.000	2,698	1,089
25.000	8,346	5,933
50.000	14,890	3,911
100.000	14,582	38,319

Remoção: Não-ordenada



Remoção: Ordenada



Interpretação: a Etapa 2 previu remover $O(n)$ no pior caso em ambas as configurações, com a mesma lógica de parada antecipada de pesquisar. Os dados confirmam o crescimento com N em ambas as séries (não há tempo constante). Há, porém, um detalhe do programa que não vem da biblioteca: olhando `removerPorTelefone` em `GerenciadorContatos`, o tempo medido cobre três percursos de lista, não um só, um `listaPorTelefone.pesquisar` (para decidir se existe e obter o objeto), depois `listaPorTelefone.remove` e `listaPorNome.remove` (cada um faz sua própria busca interna pelo nó, via comparador). Isso explica por que a remoção costuma ser

mais cara que qualquer uma das buscas isoladas da seção 3.4 no mesmo N e configuração (ex.: N=50.000 ordenada: remoção 3,911 ms contra buscas de 0,645/1,486 ms), e por que ela é mais sensível a onde o contato cai na ordenação (na ordenada, soma o custo de localizar o nó pelo telefone e pelo nome). O valor de N=100.000 ordenada (38,319 ms, bem mais alto que os demais) é coerente com o observado na seção 3.4: o telefone daquele contato específico caiu perto do topo da faixa de valores, tornando também mais caro localizá-lo (duas vezes, pelas duas listas) nesse ponto específico.

3.6 Conclusão da Etapa 3

Os testes empíricos confirmam qualitativamente as conclusões teóricas da Etapa 2, com dois pontos importantes:

1. adicionar é de fato mais barato na lista não-ordenada do que na ordenada em todos os tamanhos testados (3.3), confirmando a diferença de complexidade $O(1)$ vs. $O(n)$ apontada na Etapa 2, embora o tempo de "carregar arquivo" do programa completo não isole adicionar sozinho (3.2).
2. pesquisar/remover crescem com N em ambas as configurações (3.4 e 3.5), nenhuma das séries exibe tempo constante, confirmando que o pior caso de ambas é $O(n)$ e que a ordenação não muda a ordem de complexidade dessas operações numa lista encadeada simples (só reduz a constante na maioria dos casos, tornando as buscas/remoções na lista ordenada mais rápidas na prática, mas não mudando o crescimento assintótico).